- This lab will be a review for your upcoming midterm.
- Since this is a practice for the midterm, it is suggested that you write them out rather than try to code them.
- Think of any possible test cases that can potentially cause your solution to fail!
- If you finish early, you may leave early after showing the TA your work. Or you may stay and help other students. If you don't finish by the end of the lab, we recommend you complete it on your own time. Ideally you should not spend more time than suggested for each problem.
- Your TAs are available to answer questions in the lab, during office hours, and on Edstem about the upcoming midterm.

1. What is printed when the following Python code is executed? Draw the memory map immediately before `modify` returns and after the program finishes. That is, you should draw the variables as they are organized in the call stack, and the data they each point to (20 minutes)

```
def modify(a, b):
    a.append(4)
    b[0] = 10
    a = [5, 6]
    b = a
    b.append(7)

lst1 = [1, 2, 3]
lst2 = [8, 9]

modify(lst1, lst2)

print("lst1 =", lst1)
print("lst2 =", lst2)
```

2. For each of the following functions (loosely based on Python's built-in `list` class methods') implementations, find $f(n)$ for which the algorithm's time complexity is $\Theta(f(n))$in its **worst case** run, **identify the worst case**, and explain why. (15 minutes)

a.
```
def count(lst, value): # list.count(k)
    total = 0

    for i in range(len(lst)):
        if lst[i] == value:
            total += 1

    return total
```

b.
```
def contains(lst, value): # in list
    for i in range(len(lst)):
        if lst[i] == value:
            return True

    return False
```

c.
```
def index(lst, value): # list.index(k)
    for i in range(len(lst)):
        if lst[i] == value:
            return i

    raise ValueError
```

3. Write a recursive function that returns x raised to a nonnegative integer power n. Do not use loops, **, or pow. Make only one recursive call per non-base call. Implementation must use O(log n) arithmetic operations for n >= 2. Return 1 when n is 0, including when x is 0. (15 minutes)

```
ex) power(2, 10) returns 1024.

power(-2, 3) returns -8; power(5, 0) returns 1.

def power(x, n):
    """
    : x type: int
    : n type: nonnegative int
    : return type: int
    """
```

4. Find the missing number (30 minutes)

A list of length n contains n distinct integers from 0 through n inclusive. Exactly one number is missing. An empty list is valid and is missing 0.

(a) Analyze the worst-case runtime of the following function.

```
def find_missing_slow(lst):
    for num in range(len(lst) + 1):
        if num not in lst:
            return num
```

(b) Sorted input

Implement find_missing_sorted(lst) for an increasing list. Use O(log n) time and O(1) auxiliary space. Do not slice, sum, or sort the list. Hint: compare a value with its index.

```
find_missing_sorted([0, 1, 2, 4, 5])  # 3
find_missing_sorted([1, 2, 3])        # 0
find_missing_sorted([0, 1, 2])        # 3
find_missing_sorted([])               # 0

def find_missing_sorted(lst):
    """Return the missing int."""
    pass
```

(c) Unsorted input

Implement find_missing_unsorted(lst) in O(n) time and O(1) auxiliary space. Do not sort or modify the list.

```
find_missing_unsorted([4, 0, 2, 1])  # 3
find_missing_unsorted([])             # 0
```

```
def find_missing_unsorted(lst):
    """Return the missing int."""
    pass
```

Download the **ArrayList.py** file found under /Content/Labs on NYU Brightspace

5. Extend the ArrayList class implemented during lecture with the following methods (40 minutes) :

a. Implement delete_range(self, start, stop). Remove all elements at indices start through stop - 1, preserving the order of the remaining elements. Assume 0 <= start <= stop <= len(self). Return the number of removed elements.

   Keep the capacity unchanged and set all newly unused slots to None.

   ```
   # arr contains [10, 20, 30, 40, 50, 60]
   arr.delete_range(1, 4)   # Returns 3
   # arr contains [10, 50, 60]
   arr.delete_range(1, 1)   # Returns 0; arr is unchanged
   ```

   Shift the remaining elements in one pass. Do not repeatedly call pop or remove. Use O(n + 1) time and O(1) auxiliary space, where n is the original logical size.

   ```
   def delete_range(self, start, stop):
       pass
   ```

b. Implement __eq__(self, other) so that two ArrayList objects are equal when they contain the same number of elements and corresponding elements are equal. Their capacities do not need to match. Assume the other is an ArrayList. Do not modify either object.

```
# a contains [1, 2, 3], with capacity 4
# b contains [1, 2, 3], with capacity 8
# c contains [1, 3, 2]
a == b   # True
a == c   # False
```

Return immediately when the sizes differ or a mismatching pair is found. Use O(min(n, m) + 1) time and O(1) auxiliary space, where n and m are the two logical sizes. Assume comparing two stored elements takes constant time.

```
def __eq__(self, other):
    pass
```